

ขั้นตอนการสร้าง GUI

ขั้นตอนที่ 1 : ประกาศและสร้างวัตถุต่างๆ อาทิเช่น Frame, Panel, Components และ Listeners

```
 JButton b = new JButton("press me");
```

ขั้นตอนที่ 2 : กำหนดค่าพารามิเตอร์ต่างๆ ของวัตถุที่สร้างในขั้นตอนแรก (optional)

```
 b.text = "press me"; // avoided
```

```
 b.setText("press me");
```

ขั้นตอนที่ 3 : เพิ่มวัตถุข้างต้นลงในวัตถุอื่นๆ ตามลำดับ (bottom up) panel.add(b);

ขั้นตอนที่ 4 : ตั้งค่า listeners ให้แต่ละวัตถุ เพื่อดักฟังเหตุการณ์ที่อาจจะเกิดขึ้น

```
 // Listeners Code
```

JFrame (พื้นฐาน)

```
 public void setTitle (String str)
```

```
 กำหนดข้อความ Title ของ JFrame
```

```
 public void setSize (int width, int height)
```

```
 กำหนดความกว้างและความสูงของ JFrame
```

```
 public void setLocation (int x, int y )
```

```
 กำหนดตำแหน่งของ JFr ame บนหน้าจอ
```

```
 public void setBounds (int x, int y, int width, int height )
```

```
 กำหนดความกว้าง ความสูง และตำแหน่งของ JFrame
```

```
 public void setVisible (boolean v) * ค่าเริ่มต้น = False
```

```
 กำหนดการมองเห็น (true) หรือไม่เห็น (false) ของ JFrame
```

```
 public void setResizable ( boolean * ) เริ่มต้น = true
```

```
 กำหนดให้ปรับขนาดได้ (true) หรือไม่อนุญาตให้ปรับขนาด (false) ของ JFrame
```

```
 public void setDefaultCloseOperation (int op)
```

```
 กำหนดการกระทำที่มีเมื่อเครื่องหยาจากบาทบริเวณด้านขวาบนของ JFrame
```

```
 // window event
```

```
 - DO_NOTHING_ON_CLOSE จะไม่ทำอะไร โดยให้เรียกใช้เมธอด windowClosing ของอ็อบเจกต์ WindowListener ที่มีการลงทะเบียนไว้แล้วแทน
```

```
 - HIDE_ON_CLOSE จะทำการซ่อน JFrame หลังจากเรียกใช้อ็อบเจกต์
```

```
 ของWindowListener ที่มีการลงทะเบียนไว้แล้วแทน
```

```
 - DISPOSE_ON_CLOSE จะทำการซ่อนและทำลาย JFrame หลังเรียกใช้งานอ็อบเจกต์
```

```
 ของ WindowListener ที่มีการลงทะเบียนไว้แล้วแทน
```

```
 - EXIT_ON_CLOSE จะมีการเรียกใช้เมธอด exit ของคลาส System อ็อบเจกต์ของ
```

JPanel (พื้นฐาน)

```
 public JPanel ()
```

```
 สร้าง panel ใหม่
```

```
 public JPanel (LayoutManager layout)
```

```
 สร้าง panel ใหม่พร้อมระบุรูปแบบการจัดวาง (FlowLayout เป็นค่าเริ่มต้น)
```

```
 public void add (Component c) **ใช้บ่อย
```

```
 เพิ่ม component ใหม่ลงใน panel
```

```
 public void remove (Component c) **ใช้บ่อย
```

```
 ลบ component ออกจาก panel
```

```
 public void setLayout (LayoutManager layout) v
```

```
 กำหนดรูปแบบการจัดวาง (FlowLayout เป็นค่าเริ่มต้น)
```

```
 public void setLocation (int x, int y) v
```

```
 กำหนดตำแหน่งที่ตั้งของ JPanel บน JFrame โดยที่ตำแหน่งซ้ายบน คือ พิกัด (0,0)
```

Swing Components

```
 - JButton เป็นคลาสที่ทำหน้าที่เป็นปุ่ม
```

```
 - JLabel เป็นคลาสที่ใช้ในการสร้างอ็อบเจกต์แสดงข้อความ
```

```
 - JTextField เป็นคลาสที่ใช้ในการป้อนข้อความหนึ่งบรรทัด
```

```
 - JTextarea เป็นคลาสที่ใช้ในการป้อนข้อความหลายบรรทัด
```

```
 - JScrollBar เป็นคลาสที่ทำหน้าที่เป็นแถบควบคุมเพื่อให้ผู้ใช้เลื่อนไปยังตำแหน่งที่ต้องการ
```

```
 - JCheckBox เป็นคลาสที่ทำหน้าที่คล้ายปุ่ม
```

```
 - JChoice เป็นคลาสที่ผู้ใช้งานสามารถเลือกรายการ
```

```
 - JComboBox
```

```
String list[] = {"C","C++","C#","Java","PHP"};
JComboBox cb = new JComboBox(list);
cb.getItemAt(cb.getSelectedIndex()) -> ดึงออกมา index
cb.addItem("FewFewFew"); -> ดึงเลข
cb.setSelectedItem("C"); -> ดึงเลขมาขึ้น
```

```
 - JRadioButton
```

```
JRadioButton r1 = new JRadioButton("Male");
JRadioButton r2 = new JRadioButton("Female");
ButtonGroup bg = new ButtonGroup();
bg.add(r1); bg.add(r2); -> ดึงมาจับ ไม่เลือกได้ทั้งคู่
```

```
 - JOptionPane
```

```
JOptionPane.showMessageDialog(
null, "Message", "Title", JOptionPane.PLAIN_MESSAGE);
```

```
• JOptionPane.ERROR_MESSAGE
• JOptionPane.PLAIN_MESSAGE
• JOptionPane.QUESTION_MESSAGE
• JOptionPane.INFORMATION_MESSAGE
• JOptionPane.WARNING_MESSAGE
```

```
 - JCheckBox
```

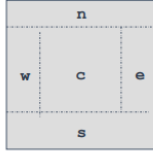
```
ck = new JCheckBox("Check me!"); ck.setSelected(true);
boolean selected = ck.isSelected();
```

ตัวอย่างการสร้าง GUI

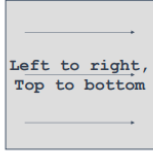
```
import javax.swing.*;
public class GUI01 {
    public static void main(String[] args){
        // step 1 : create
        JFrame f = new JFrame("title");
        JPanel p = new JPanel() ;
        JButton b = new JButton("press me");
        // step 2 : config
        f.setSize(200, 200);
        // step 3 : add
        p.add(b); // add button to panel
        f.add(p); // add panel to frame
        // step 4 : Listener
    }
}
```

การจัดตำแหน่ง (Layout)

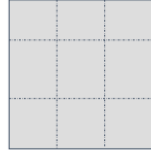
BorderLayout



FlowLayout



GridLayout



BorderLayout (ต้องระบุตำแหน่ง)

```
 - แบ่งพื้นที่ container 5 zone
```

```
 - แต่ละ zone ขนาดไม่เท่ากัน
```

```
 - เวลา add ลงแต่ละ zone component จะโดนยืดให้เท่ากับ zone
```

FlowLayout

```
 - ไม่แบ่ง zone
```

```
 - เรียงจากซ้ายไปขวา บนไปล่าง
```

```
 - component จะไม่ถูกยืด
```

GridLayout

```
 - แบ่ง zone เป็น N zone (ในรูปแบบตาราง)
```

```
 - แต่ละ zone มีขนาดเท่ากัน
```

```
 - เวลา add ลงแต่ละ zone component จะโดนยืดให้เท่ากับ zone
```

เหตุการณ์ (Event)



```
 - Event Object มีหลายตัว
```

```
 - Event Object แต่ละตัวถูกสร้างขึ้นมาโดยมีเงื่อนไขอย่างไร ?
```

WindowEvent (เพิ่มเติม)

```
 public void windowOpened(WindowEvent we){}
```

```
 public void windowClosed(WindowEvent we){}
```

```
 public void windowClosing(WindowEvent we){}
```

```
 public void windowIconified(WindowEvent we){}
```

```
 public void windowDeiconified(WindowEvent we){}
```

```
 public void windowActivated(WindowEvent we){}
```

```
 public void windowDeactivated(WindowEvent we){}
```

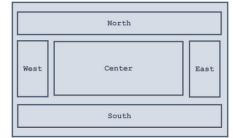
Note ArrayList

```
ถ้าสมมติมีค่าตัวเดียวควรประกาศ
Generic ด้วย Example.
ArrayList<String> , ArrayList<Book>
```

ตัวอย่างการสร้าง JFrame

```
import javax.swing.*;
public class GUI02 {
    public static void main(String[] args){
        // JFrame frame = new JFrame();
        JFrame frame = new JFrame("Title GUI 02");
        JLabel lable = new JLabel("Faculty of Information Technology");

        frame.add(lable);
        // อยู่ล่างสุดเสมอ
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.pack(); // auto adjust
        frame.setVisible(true); // can see (default=false)
    }
}
```



```
frame.add(components, BorderLayout.ทิศ)
```

WARNING !

- สร้าง ก่อน เพิ่ม (Components) ที่หลัง
- import java.awt.*; , javax.swing.*; , java.awt.event.*;
- implements ActionListener, WindowListener
- .addActionListener, WindowListener;
- ห้ามลืม @Override
- ไม่รู้อะไร = Press ALT + Enter หรือ ctrl + spacebar
- อย่าลืมสวดมนต์ก่อนสอบ เลือกเก้าอี้เข้าห้อง
- สำคัญมาก ! >> อย่าลืมยามดม + ปากกา + บัตรนศ.



หายใจลึกๆ นะครีนักศึกษา
อ.ธราวิชญ์ ธิติจรูญโรจน์

ActionEvent

- คลิกเมาส์บน JButton
- ป้อนคีย์ Enter ใน JTextField
- เลือกเมนูใน JMenuItem
- กด double คลิกใน JList

```
public void actionPerformed(ActionEvent e){
    statements;
}
```

//MouseListener

```
public void mouseClicked(MouseEvent me){}
public void mouseEntered(MouseEvent me){}
public void mouseExited(MouseEvent me){}
public void mousePressed(MouseEvent me){}
public void mouseReleased(MouseEvent me){}
//MouseMotionListener
public void mouseMoved(MouseEvent me){}
public void mouseDragged(MouseEvent me){}
```

Array & Collections

```
Array >> ตัวแปรชนิด อ้างอิง ที่เก็บข้อมูลชนิดเดียวกันได้
```

```
 - มีเครื่องหมาย [] อยู่หน้า/หลังตัวแปรก็ได้ ex. int []num; หรือ int num[];
```

```
 - การสร้าง Array >> num = new int [4];
```

```
 - สามารถกำหนดค่าให้สมาชิกแต่ละตัวใน Array ได้ variableName[idx] = value;
```

```
List >> value เข้ากันได้ มีลำดับความสำคัญ
```

```
Array List >> ดึงออกมาจาก Array ปรับขนาดได้ เข้าถึงข้อมูลได้แบบ Direct
```

```
**Extra Array List insert ข้อมูล เข้าใน ArrayList เข้าเ้าคลาสที่สร้างขึ้นมา
```

```
private ArrayList<Book> data = Data.data;
if (e.getSource().equals(insertButton)) {
    if (!nameTextField.getText().isEmpty() ||
        priceTextField.getText().isEmpty()) {
        String name = nameTextField.getText();
        Double price = Double.parseDouble(priceTextField.getText());
        String type = (String) TypeComboBox.getSelectedItem();

        Book book = new Book(name, price, type);
        data.add(book);
    }
}
```

Event Handler

```
จัดการกับเหตุการณ์โดยอาศัย Interface ประเภท Listener
```

Mouse Event

- Pressed >> คลิกเมาส์ค้าง
- Released >> ปล่อยเมาส์
- Dragged >> ลากเมาส์
- Moved >> ขยับเมาส์
- Clicked >> คลิกเมาส์
- entered >> เลื่อนเมาส์ไปบน component
- Exited >> เลื่อนเมาส์ออกจาก component

What is ข้อผิดพลาด

- Error >> แก้ไขไม่ได้ มักเกิดใน Runtime
- Exception >> แก้ไขได้ มักเกิดใน Runtime หรือ compile time

Type of Exception

- ArithmeticException - คิดเลขผิด คิดไม่ได้ เช่น 1/0
- ArrayIndexOutOfBoundsException - เกินขอบเขตของ Array
- NullPointerException - ตัวแปรนั้นมีค่าเป็น null
- NumberFormatException - ไม่สามารถเป็นตัวเลขได้
- EOFException - อ่านไฟล์หมดแล้ว แต่ยังอ่านต่อ
- FileNotFoundException - ไม่เจอไฟล์
- IOException - พิลแบบบันทึกไม่ได้ อ่านไม่ได้ (อ่านข้อความเอา)
- InterruptedException - เกิดขึ้นเวลาทำงานหลาย Thread แล้วเวลาล้าง sleep แต่อีก Thread มาสั่งห้าม sleep จะเกิด InterruptedException

การจัดการ Exception

คำสั่ง Try - catch - finally

```
try () {
    statements;
} catch (Exception) {
    statements;
} finally { <----- ช่วงสุดท้ายก่อนออกจาก Try - catch
    statements;
}
```

คำสั่ง Try - With - Resource

```
try (resource declaration) {
    //statement;
}
```

คำสั่ง Rethrowing >> การโยนให้ throw แบบไม่เปลี่ยนประเภท exception

คำสั่ง Wrapping >> การโยนให้ throw แบบเปลี่ยนให้มีค.ชัดเจนมากขึ้น

```
public void method () throws MyOwnException, NewException {
    try {
        system.out.println ("Hello world");
    }
    catch (NewException ex) { // wrapping
        throw new NewException ("Bye Bye");
    }
    catch (MyOwnException ex) { throw e; } // Rethrowing
}
```

การสร้าง Exception ใหม่

```
public class MyOwnException extends Exception {
    public MyOwnException (String s) {
        super(s);
    }
}
```

คำสั่ง Throws >> สามารถมีได้หลายตัว ใส่ที่ method

คำสั่ง Throw >> สามารถมีได้หลายตัว ใส่ใน method

```
public void method() throws MyOwnException {
    throw new MyOwnException ("My exception ");
}
```

โครงสร้างช่องทางการสื่อสาร



ประกอบด้วย 3 ส่วนหลัก ได้แก่ ต้นทาง (input source) ช่องทางการแลกเปลี่ยนข้อมูล (stream) และปลายทาง (output sink) ปลายทางอาจจะเป็น ฮาร์ดแวร์หรือซอฟต์แวร์ เช่น ไฟล์ หน่วยความจำ หรือ socket สำหรับภาษาจาก stream มีทั้งหมด 2 ประเภท ได้แก่ byte stream และ character stream อีกชื่อ reader หรือ writer

โหนดสำหรับ Stream

ไฟล์ คือ โหนดสำหรับ stream ที่เป็นไฟล์สำหรับอ่านหรือเขียนข้อมูลชนิด byte หน่วยความจำ คือ โหนดสำหรับ stream ที่ใช้สำหรับอ่านหรือเขียนข้อมูลที่เป็น Array ไปป์ คือ โหนดสำหรับ stream ที่จะส่งหรือรับข้อมูลระหว่าง process หรือโปรแกรมเรต String

Thread implements Runnable

- static void sleep(long mills)
- เป็นเมธอดที่กำหนดแบบเรตที่อยู่ในสถานะ Running หยุดทำงานเป็นเวลา mills มิลลิวินาที
- boolean isAlive()
- เป็นเมธอดใช้ตรวจสอบว่าอปเจ็คแบบเรตยังมึสถานะ Running อยู่หรือไม่
- void setPriority(int p)
- เป็นเมธอดใช้ในการกำหนดลำดับความสำคัญของอปเจ็คแบบเรต ซึ่งจะมีค่าระหว่าง 1 ถึง 10 (ค่าสูงสุด)
- int getPriority()
- เป็นเมธอดที่ใช้ในการเรียกดูค่าลำดับความสำคัญของอปเจ็คแบบเรต
- void run()
- เป็นเมธอดที่จะถูกเรียกใช้เมื่อ CPU รันโปรแกรมอปเจ็คแบบเรต
- void start()
- เป็นเมธอดเพื่อกำหนดให้ออปเจ็คแบบเรต เริ่มทำงานโดยเข้าสู่สถานะ Runnable
- Void suspended()
- เป็นเมธอดใช้ในการหยุดทำงานอปเจ็คแบบเรต
- void resume()
- เป็นเมธอดใช้สั่งให้ออปเจ็คแบบเรตกลับเข้าสู่สถานะ Runnable

```
import java.awt.*;
import javax.swing.*;
public class CalculatorTwoGUI {
    private JFrame fr;
    private JPanel p1;
    private JPanel p2;;
    private JTextField txt;

    public CalculatorTwoGUI(){
        fr = new JFrame(title: "My Calculator");
        fr.setSize(width: 400,height: 400);
        p1 = new JPanel();
        p2 = new JPanel();
        txt = new JTextField();
        p1.add(comp: txt);
        p1.setLayout(new GridLayout(rows:1, cols:1))
        p2.add(new JButton(text:"7"));
        p2.add(new JButton(text:"8"));
        p2.add(new JButton(text:"9"));
        p2.add(new JButton(text:"+"));
        p2.add(new JButton(text:"4"));
        p2.add(new JButton(text:"5"));
        p2.add(new JButton(text:"-"));
        p2.add(new JButton(text:"1"));
        p2.add(new JButton(text:"2"));
        p2.add(new JButton(text:"3"));
        p2.add(new JButton(text:"x"));
        p2.add(new JButton(text:"0"));
        p2.add(new JButton(text:"c"));
        p2.add(new JButton(text:"="));
        p2.add(new JButton(text:"/"));
        p2.setLayout(new GridLayout(rows:4, cols:4));

        fr.add(comp: p1, constraints:BorderLayout.NORTH);
        fr.add(comp: p2, constraints:BorderLayout.CENTER);
        fr.setDefaultCloseOperation(operation: JFrame.EXIT_ON_CLOSE);
        fr.setVisible(b: true);
    }
}
```

บันทึกข้อมูลลงในไฟล์

*ก่อนบันทึกข้อมูลเราา Object ตามนี้

Must implement Serializable!

```
try (FileOutputStream stream = new FileOutputStream(new File("Few.data"))
ObjectOutputStream outputStream = new ObjectOutputStream(stream);
outputStream.writeObject(data);
} catch (IOException ex) {
    ex.printStackTrace();
}

Object Stream
```

- คลาส ObjectOutputStream เป็นคลาสที่ใช้ในการเขียนข้อมูลใด ๆ ลงใน stream โดยมันจะ write(Object) ที่ใช้ในการเขียนข้อมูล
- ส่วนคลาส ObjectOutputStream เป็นคลาสที่ใช้ในการอ่านข้อมูลใด ๆ มาจาก stream โดยมันจะ read(Object) ที่ใช้ในการอ่านข้อมูล

การใช้ File writer

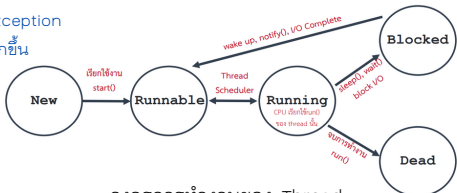
```
public void saveFile(String content) {
    try (FileWriter fw = new FileWriter(currentSelectFile)) {
        for (int i = 0; i < content.length(); i++) {
            fw.write(content.charAt(i));
        }
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}

การใช้ File reader
```

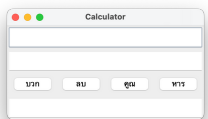
```
public String openFile(File file) {
    this.currentSelectFile = file;
    String text = "";
    try (FileReader fileReader = new FileReader(currentSelectFile)) {
        int ch;
        while ((ch = fileReader.read()) != -1) {
            text += (char) ch;
        }
    } catch (IOException ex) {
        ex.printStackTrace();
    }
    return text;
}
```

Collection Method

- boolean add (Object element)
- boolean remove (Object element)
- int size()
- boolean isEmpty()
- boolean contains (Object element)
- Iterator iterator()



วงจรการทำงานของ Thread

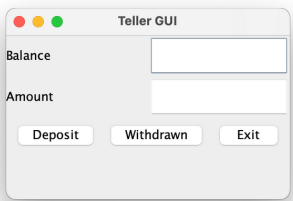


```
import java.awt.*;
import javax.swing.*;
public class CalculatorOneGUI{
    private JFrame fr;
    private JPanel p;
    private JLabel lbl1;
    private JTextField txt1;
    private JTextField txt2;
    private JButton btn1;
    private JButton btn2;
    private JButton btn3;
    private JButton btn4;
    private JTextField txt3;

    public CalculatorOneGUI(){
        fr = new JFrame(title: "Calculator");
        p = new JPanel();
        txt1 = new JTextField();
        txt2 = new JTextField();
        btn1 = new JButton(text:"บวก");
        btn2 = new JButton(text:"ลบ");
        btn3 = new JButton(text:"คูณ");
        btn4 = new JButton(text:"หาร");
        txt3 = new JTextField();

        fr.setLayout(new GridLayout(rows:4,cols:1));
        fr.add(comp: txt1);
        fr.add(comp: txt2);
        p.add(comp: btn1); p.add(comp: btn2); p.add(comp: btn3); p.add(comp: btn4);
        fr.add(comp: p);
        fr.add(comp: txt3);

        fr.setDefaultCloseOperation(operation: JFrame.EXIT_ON_CLOSE);
        fr.pack();
        fr.setVisible(b: true);
    }
}
```



```
import javax.swing.*;
import java.awt.*;

public class TellerGUI {

    private JFrame fr;
    private JPanel p1, p2, p3;

    public TellerGUI() {
        fr = new JFrame(title: "Teller GUI");
        p1 = new JPanel();
        p2 = new JPanel();
        p3 = new JPanel();
        fr.setSize(width: 300, height: 200);
        p1.add(new JLabel(text:"Balance"));
        p1.add(new JTextField());
        p1.setLayout(new GridLayout(rows:1, cols:2));

        p2.add(new JLabel(text:"Amount"));
        p2.add(new JTextField());
        p2.setLayout(new GridLayout(rows:1, cols:2));

        p3.add(new JButton(text:"Deposit"));
        p3.add(new JButton(text:"Withdrawn"));
        p3.add(new JButton(text:"Exit"));

        fr.add(comp: p1);
        fr.add(comp: p2);
        fr.add(comp: p3);

        fr.setLayout(new GridLayout(rows:4, cols:1));
        fr.setDefaultCloseOperation(operation: JFrame.EXIT_ON_CLOSE);
        fr.setVisible(b: true);
    }
}
```